

Rawshan Sinde

CSC 3130-02

03/12/2026

Assignment 4

1. Given an empty stack, the contents of the stack after each of the following operations will be:
 - a. $\text{push}(1) = 1$ will be placed at the top of the stack but since the stack was empty before, it is the only node in the stack so far. The resulting stack is 1.
 - b. $\text{push}(20) = 20$ will be pushed on top of 1 resulting in a stack where the top is 20 and the bottom is 1. The resulting stack is 20, 1.
 - c. $\text{pop}()$ = removes the top node and returns a local variable with the top node's data. The resulting stack will be 1.
 - d. $\text{push}(\text{pop()}*2)$ = first the $\text{pop}()$ operation removes the top node from the stack and returns a local variable of 1, then we multiple $1 * 2$ which equals 2, and push the value 2 into the stack. The resulting stack is 2.
 - e. $\text{push}(0)$ = placing 0 at the top of the stack; The resulting stack is 0, 2 where 0 is the top and 2 is the bottom.
 - f. $\text{push}(\text{pop}()/2)$ = first we use $\text{pop}()$ which removes the top node from the stack and returns a local variable of 0. $0/2 = 0$, now we push 0 to the top of the stack. The resulting stack is 0, 2 with 0 as top and 2 as bottom.
2. Given an empty queue, the contents of the stack after the following operations

3. An algorithm that can find the position in a deque in $O(n/2)$ would start with a deque that contains n elements, because a deque allows access from both the front and the back, we can search from both ends at the same time.
 - a. Let $i = 0$
 - b. While $i \leq n/2$, check the i -th element from the left and the right
 - c. If either element equals y , return its position
 - d. Increase i and continue

Instead of searching sequentially from only one side, the algorithm checks two elements during each iteration, one from the left and one from the right. This ensures that the search progresses towards the center of the deque from both directions simultaneously.

The complexity is $O(n/2)$ where n is the number of elements in the deque because the algorithm examines two elements per iteration. Since each iteration checks two elements, the algorithm requires at most $n/2$ iterations to inspect all elements in the deque.

7a) The time complexity of `BalancedSymbols` is $O(n)$ where n is the length of the string because the algorithm iterates through the string once and performs only constant-time stack operations for each character. The space complexity is $O(n)$ where n is the length of the string, because in the worst case every character could be pushed onto the stack, causing the stack to store up to n elements at one time.

7b) The time complexity of `DecodeStrings` is $O(n \times k)$ where n is the length of the string and k is the repetition count, because the algorithm iterates through the string once and may repeat parts of the string multiple times inside the inner loop when building the

decoded result. The space complexity is $O(n)$ where n is the length of the string because the stacks may store up to n elements while the program processes nested brackets.

7c) The time complexity of InfixToPostfix is $O(n)$ where n is the length of the string because the algorithm iterates through the string once and each character is pushed to or popped from the stack at most one time. The space complexity is $O(n)$ where n is the length of the string because in the worst case, the stack can store up to n characters.